helix_ota

Helix OTA

We have created new repo for our project: https://github.com/HelixDevelopment/helix_ota Here we MUST implement Go lang application / system which will be the OTA server + client libs, sdks and apps so devices who incorporate the server and required client side components can perform OTA updates. This system MUST BE universal, generic, decoupled to smallest details so we can easy in any moment incorporate support for new operating systems, and new versions of the systems. Starting point for us is to support fully OTA updates system for Android 15 - all flavors and variants of it. We have our own Android 15 based System running on Orange Pi 5 Max and we want (we MUST) incorporate into it full OTA updating capabilities. Our build pipelines produces for us flashing images and OTA update zip files with mandatory hash files for verification. We MUST incorporate this so we can easily login to the System dashboard (safely) and upload new zip file. On other side, our Android 15 based System MUST detect (check on every X minutes for the new updates) the update, download it, validate and verify and safely install it! There MUST NOT be possibility for corruption of the System, breaking working things and so on! If possible we MUST have possibility for end ushers to rollback to rollback to one of versions from before. This has to be researched, can be done in later phases. Not for first version required! Process of uploading of OTA update zip file MUST BE safe as well so all mandatory validation steps MUST BE performed! Once new OTA update is uploaded we MUST BE able to rollout this uploaded update to: all subscribers at once, partially - by 5%, 10%, 30%, and son, to all what is left up to 100%. System MUST HAVE mechanism for tracking and measuring and obtaining critical data, to detect problems and to report. We MUST define for this project what MUST be MVP, what version 1.0.0, what goes into 1.0.1 and so on. For all this we must organize the whole research materials into directories per the phase: 1.0.0-mvp, 1.0.1-some_name, and so on. Inside we MUST HAVE everything required for this implementation to happen! All technical documentation! Whole code snippets and whole components developed! Up to the fine grained and refined methods which will be used in real implemented product! We MUST HAVE all documentation + testing strategies, diagrams, schemes, SQL definitions, and all this ready for the development team to take it and implement it all fully! We MUST DO deep web research for any existing System for OTA updating which we can wrap up / encapsulate under our hood! If there are MOST POPULAR, MOST STABLE, MOST FEATURE COMPLETE solutions which are opensourced, we shall make it wrapped up using our containers Submodule: https://github.com/vasic-digital/containers (git@github.com:vasic-digital/containers.git). All required infrastructure MUST BE as well fully containerized! Use all developed Submodules (and mandatory dependencies for them) from our vasic-digital (https://github.com/orgs/vasic-digital/repositories) and HelixDevelopment (https://github.com/orgs/HelixDevelopment/repositories) organizations. We have already developed many

reusable System building bricks which you can incorporate into this comprehensive development plan! Dive deep into each repository and see where and if they can be used and if they fir for our needs. We can add to them any missing features or functionalities in the area / context of work they are offering to us! We MUST make sure that we all reusable building bricks, services, components decouple heavily so they can be used in future projects as well! Since we have full access to GitHub and GitLab CLIs we MUST create for any new Submodules repositories for both GitHub and GitLab under our organizations (ALL OF THEM MUST BE PUBLIC) to which newly created repos for Submodule can be pushed! Every new Submodule is reusable individual project which MUST HAVE in depth documentation, user guides, manuals, diagrams, schemes and all other relevant documentation! Everything MUST BE covered with the tests! We MUST follow in planning all mandatory rules and constraints from our main constitution Submodule which MUST BE included into the project completely: https://github.com/HelixDevelopment/HelixConstitution (git@github.com:HelixDevelopment/HelixConstitution.git). We expect from you research in-depth comprehensive work in hundreds of files and thousands of pages per file! Make widest, deepest up to nano detail planning! Nothing can be left out, not a single detail! We cannot skip anything! Research MUST provide real stuff! Real results with bluff(s) of any kind! Make sure that every part of the research and produced documentation and materials is reviewed (code reviewed) in depth by code review subagents! we MUST detect and prevent any shortcomings, bottlenecks, danger zones and possible show stoppers! keep in mind that this MUST BE enterprise grade cutting edge solution which will scale! Leave planned and open possibility that after supporting all variants and flavors of Android we will continue with ultimate support for Linux distributions - all types and flavors, then Microsoft Windows, and all other operating Systems that are there and that are upstreams! We MUST create at least basic planning and research for all of them put in proper directory for future expansions and versions - 1.X.X-some_name, etc! if there are multiple standards for particular OS and its version(s) and flavors WE MUST SUPPORT it all! We MUST all documentation files (markdown format) have exported in the following formats as well: PDF, HTML, DOCX. All diagrams and schemes or drawings into: mermaid js, draw .io, svg, uml, html, png. Perform as much as needed iterations, spawn as much as needed subagents to make this ultimate complete project plan / documentation ever made! No holes, gaps, imprefections, misunderstanding, false results or bluff of any kind is allowed!!!